
Incluyendo reglas de integridad (*Constraints*)

Objetivos

En este capítulo se revisará lo siguiente:

- La descripción de las reglas de integridad (*constraints*)
- La creación y administración de los *constraints*

¿Qué es un *Constraints*?

El servidor de Oracle utiliza *constraints* para prevenir el registro de datos no válidos a las tablas.

Se pueden utilizar los *constraints* para lo siguiente:

- Implementar o imponer reglas en los datos de una tabla cuando una fila es insertada, modificada o borrada de la tabla. El *constraint* se debe cumplir para que la operación se realice.
- Previene la eliminación de una tabla si existen dependencias con otras
- Proporciona reglas para herramientas de Oracle, como *Oracle Developer*

Tipos de *Constraints*

<i>Constraint</i>	Descripción
NOT NULL	Especifica que la columna no puede contener un valor nulo
UNIQUE	Especificado para una columna o combinación de columnas cuyos valores deben ser únicos para todas las filas de la tabla
PRIMARY KEY	Identifica de manera única cada fila de una tabla
FOREIGN KEY	Establece e impone una relación entre una o más columnas de la tabla y una o más columnas de la tabla referenciada
CHECK	Especifica una condición que se debe cumplir

Reglas en el uso de *constraints*

Todos los *constraints* son almacenados en el diccionario de datos. Los *constraints* son fáciles de referenciar si les proporcionas un nombre adecuado. Los *constraints* deben seguir el estándar para nombrar objetos. Si tu no especificas un nombre para tu *constraint*, el servidor de Oracle genera un nombre con el siguiente formato **SYS_Cn**, donde **n** es un entero que identifica el nombre del *constraint* como único.

Los *constraints* pueden ser definidos al mismo tiempo que se crea una tabla o después de que la tabla ha sido creada.

Se pueden ver los *constraints* definidos para una tabla específica en la tabla USER_CONSTRAINTS del diccionario de datos.

Defining Constraints

```
CREATE TABLE [schema.]table
  (column datatype [DEFAULT expr]
   [column_constraint],
   ...
   [table_constraint] [,...]);
```

```
CREATE TABLE employees(
  employee_id  NUMBER(6),
  first_name   VARCHAR2(20),
  ...
  job_id       VARCHAR2(10) NOT NULL,
  CONSTRAINT emp_emp_id_pk
             PRIMARY KEY (EMPLOYEE_ID));
```

ORACLE

Definiendo constraints

En la imagen anterior se muestra la sintaxis para definir *constraints* en la creación de una tabla.

Donde:

<i>schema</i>	es el mismo que el nombre del propietario
<i>table</i>	es el nombre de la tabla
<i>DEFAULT exp.</i>	especifica un valor por defecto para usar si un valor es omitido en la sentencia <i>INSERT</i>
<i>column</i>	es el nombre de la columna
<i>datatype</i>	es el tipo de dato y longitud de la columna
<i>column_constraint</i>	es un <i>constraint</i> de integridad que forma parte de la definición de la columna
<i>table_constraint</i>	es un <i>constraint</i> de integridad que forma parte de la definición de la tabla

Los *constraints* son usualmente creados al mismo tiempo que la tabla, pero pueden ser añadidos después de la creación de la misma y también pueden ser deshabilitados temporalmente.

Los *constraints* pueden estar definidos en uno de dos niveles que son:

Nivel de <i>Constraint</i>	Descripción
Columna	Se refiere a una sola columna y es definida en la especificación de la columna; se puede definir cualquier tipo de <i>constraint</i> <i>column</i> [<i>CONSTRAINT constraint_name</i>] <i>constraint_type</i> ,
Tabla	Se refiere a una o mas columnas y es definida fuera de las especificaciones de las columnas de una tabla; se puede definir cualquier tipo de <i>constraint</i> excepto NOT NULL <i>column</i> ,... [<i>CONSTRAINT constraint_name</i>] <i>constraint_type</i> (<i>column</i> , ...),

Donde:

constraint_name

es el nombre del *constraint*

constraint_type

es el tipo de *constraint*

The NOT NULL Constraint

Ensures that null values are not permitted for the column:

EMPLOYEE_ID	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	DEPARTMENT_ID
100	King	SKING	515.123.4567	17-JUN-87	AD_PRES	24000	90
101	Kochhar	NKOCHHAR	515.123.4568	21-SEP-89	AD_VP	17000	90
102	De Haan	LDEHAAN	515.123.4569	13-JAN-93	AD_VP	17000	90
103	Hunold	AHUNOLD	590.423.4567	03-JAN-90	IT_PROG	9000	60
104	Ernst	BERNST	590.423.4568	21-MAY-91	IT_PROG	6000	60
178	Grant	KGRANT	011.44.1644.429263	24-MAY-99	SA_REP	7000	
200	Whalen	JWHALEN	515.123.4444	17-SEP-87	AD_ASST	4400	10

...
20 rows selected.



NOT NULL constraint
(No row can contain a null value for this column.)



NOT NULL constraint



Absence of NOT NULL constraint
(Any row can contain null for this column.)

ORACLE

Constraint NOT NULL

El *constraint NOT NULL* se asegura de que las columnas no contengan valores nulos. Las columnas sin un *constraint NOT NULL* pueden contener valores nulos por defecto.

The NOT NULL Constraint

Is defined at the column level:

```
CREATE TABLE employees(
  employee_id    NUMBER(6),
  last_name      VARCHAR2(25) NOT NULL,
  salary         NUMBER(8,2),
  commission_pct NUMBER(2,2),
  hire_date      DATE
  CONSTRAINT emp_hire_date_nn
  NOT NULL,
  ...
)
```

← System named

← User named

ORACLE

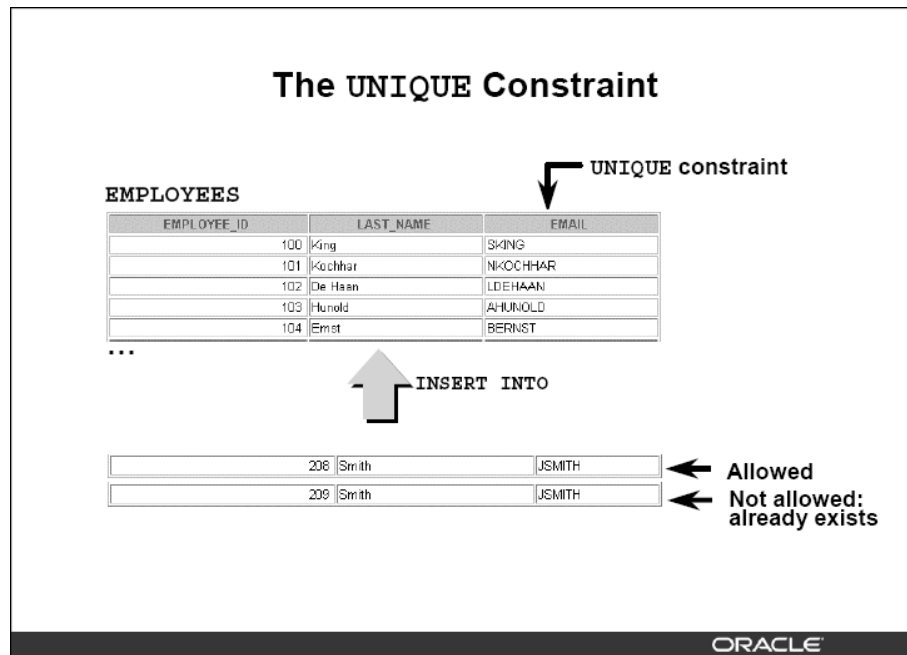
El *constraint NOT NULL* puede ser especificado solamente a nivel de columnas y no a nivel de tabla.

En el ejemplo anterior se aplica un *constraint NOT NULL* a las columnas LAST_NAME y HIRE_DATE de la tabla EMPLOYEES. Debido a que esos *constraints* no tienen nombre, el servidor de Oracle crea nombres para ellos.

Se puede especificar el nombre del *constraint* cuando se especifica:

```
... last_name VARCHAR2(25)
  CONSTRAINT emp_last_name_nn NOT NULL...
```

Nota: Los *constraints* de ejemplo descritos en este capítulo, pueden no estar presentes en las tablas utilizadas para el curso. Si es requerido, estos *constraints* pueden ser añadidos a las tablas.



Constraint UNIQUE

Un *constraint UNIQUE* requiere que todos los valores en una columna o conjunto de columnas sean únicos- esto es, dos filas de una tabla no pueden tener valores duplicados en la columna o conjunto de columnas especificadas. La columna (o conjunto de columnas) incluida en la definición del *constraint UNIQUE* es llamada *llave única (unique key)*. Si el *constraint UNIQUE* comprende mas de una columna, el grupo de columnas es llamado *llave única compuesta (composite unique key)*.

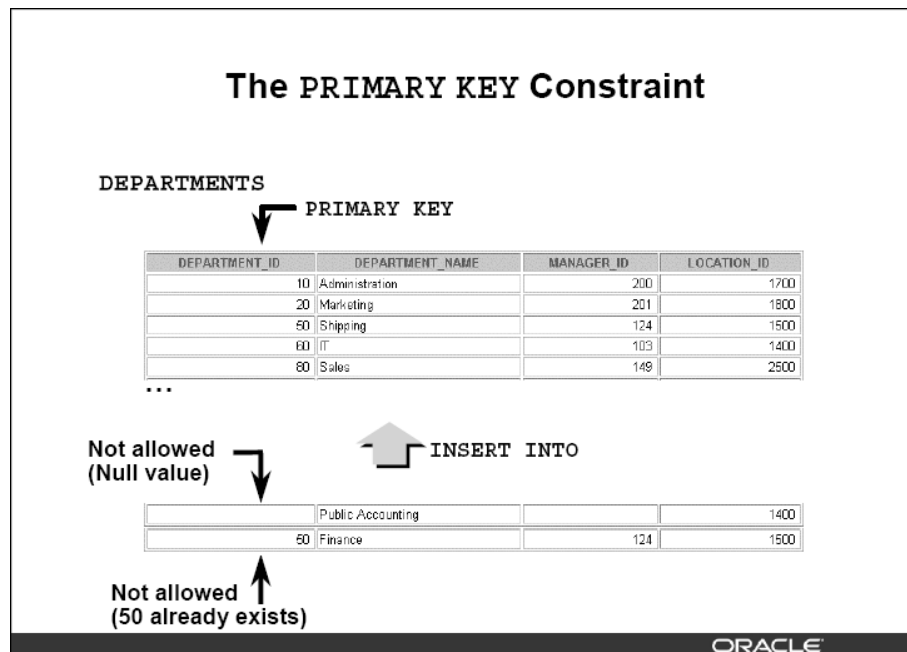
Los *constraints UNIQUE* permiten el uso de valores nulos siempre y cuando no se hayan definido *constraints NOT NULL* para las mismas columnas. De hecho, cualquier número de filas puede incluir nulos para las columnas sin *constraints NOT NULL* puesto que los valores nulos no son iguales a nada. Un valor nulo en una columna (o en todas las columnas de una llave única compuesta) siempre satisface un *constraint UNIQUE*.

Un *constraint UNIQUE* puede ser definido a nivel de columna o tabla. Una llave única compuesta es creada usando el nivel de definición de tabla.

```
CREATE TABLE employees(
  employee_id      NUMBER(6),
  last_name        VARCHAR2(25) NOT NULL,
  email            VARCHAR2(25),
  salary           NUMBER(8,2),
  commission_pct   NUMBER(2,2),
  hire_date        DATE NOT NULL,
  ...
  CONSTRAINT emp_email_uk UNIQUE(email));
```

En el ejemplo anterior se aplica un *constraint UNIQUE* a la columna EMAIL de la tabla EMPLOYEES. El nombre del *constraint* es EMP_EMAIL_UK.

Nota: El servidor de Oracle forza a que el *constraint UNIQUE* implícitamente cree un índice único para la columna o columnas.



Constraint PRIMARY KEY

Un *constraint PRIMARY KEY* crea una llave primaria para la tabla. Solo una llave primaria puede ser creada por cada tabla. El *constraint PRIMARY KEY* es una columna o conjunto de columnas que identifica de forma única cada fila de una tabla. Estos *constraints* obligan valores únicos para la columna o combinación de columnas y aseguran que estas columnas no puedan contener valores nulos.

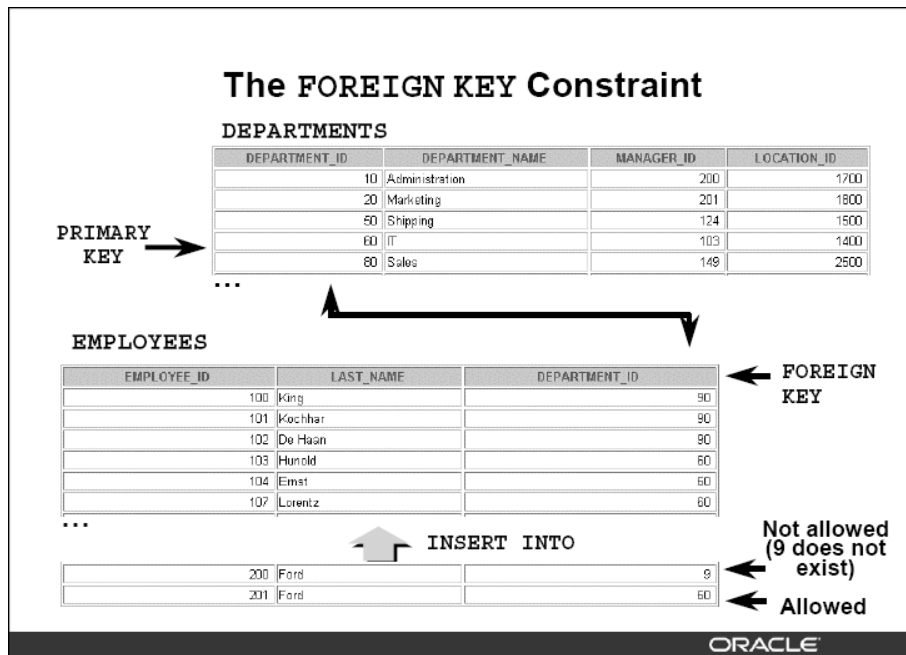
Los *constraints PRIMARY KEY* pueden ser definidos a nivel de columna o a nivel de tabla. Una *llave primaria compuesta* es creada usando la definición a nivel de tabla.

Una tabla puede tener solo un *constraint PRIMARY KEY* pero puede tener diversos *constraints UNIQUE*.

```
CREATE TABLE departments(
    department_id      NUMBER(4),
    department_name    VARCHAR2(30)
        CONSTRAINT dept_name_nn NOT NULL,
    manager_id        NUMBER(6),
    location_id        NUMBER(4),
    CONSTRAINT dept_id_pk PRIMARY KEY(department_id));
```

En el ejemplo anterior se define un *constraint PRIMARY KEY* en la columna DEPARTMENT_ID de la tabla DEPARTMENTS. El nombre del *constraint* es DEPT_ID_PK.

Nota: Un índice único es automáticamente creado para la columna de llave primaria.



Constraint FOREIGN KEY

Un *Foreign Key* o *llave foránea*, es un *constraint* de integridad referencial que designa a una columna o combinación de columnas como una *llave foránea* estableciendo una relación entre una *llave primaria* o *llave única* en la misma tabla o en una tabla diferente. En el ejemplo anterior, DEPARTMENT_ID está definida como *llave foránea* en la tabla EMPLOYEES (tabla dependiente o hija); esta referencia de la columna DEPARTMENT_ID de la tabla DEPARTMENTS es la tabla referenciada o tabla padre.

Un valor de *llave foránea* debe corresponder con un valor existente en la tabla padre o ser nulo.

Las *llaves foráneas* están basadas en los valores de los datos y su lógica pura, no física o apuntadores.

```
CREATE TABLE employees(
  employee_id      NUMBER(6),
  last_name        VARCHAR2(25) NOT NULL,
  email            VARCHAR2(25),
  salary           NUMBER(8,2),
  commission_pct   NUMBER(2,2),
  hire_date        DATE NOT NULL,
  ...
  department_id    NUMBER(4),
  CONSTRAINT emp_dept_fk FOREIGN KEY (department_id)
    REFERENCES departments(department_id),
  CONSTRAINT emp_email_uk UNIQUE(email));
```

Un *constraint FOREIGN KEY* puede ser definido a nivel de columna o a nivel de tabla. Una *llave foránea* compuesta debe ser creada utilizando una definición a nivel de tabla.

En el ejemplo anterior se define un *constraint FOREIGN KEY* en la columna DEPARTMENT_ID de la tabla EMPLOYEES, utilizando la sintaxis a nivel de tabla. El nombre del *constraint* es EMP_DEPTID_FK, La *llave foránea* también puede ser definida a nivel de columna, puesto que el *constraint* está basado en una columna. La sintaxis difiere puesto que la palabra reservada FOREIGN KEY no aparece, por ejemplo:

```
CREATE TABLE employees
(
...
department_id NUMBER(4) CONSTRAINT emp_deptid_fk
REFERENCES departments(department_id),
...
)
```

La *llave foránea* esta definida en la tabla hija, y la tabla conteniendo la columna referenciada es la tabla padre. La *llave foránea* es definida al utilizar una combinación de las siguientes palabras reservadas:

- FOREIGN KEY es usada para definir la columna en la tabla hija como un *constraint* a nivel tabla
- REFERENCES identifica la tabla y columna en la tabla padre
- ON DELETE CASCADE indica que cuando la fila en la tabla padre es eliminada, las filas dependientes en la tabla hija pueden ser eliminadas.
- ON DELETE SET NULL convierte los valores de las *llaves foráneas* a nulos cuando el valor del padre es removido

El comportamiento por defecto es llamado regla de restricción, cuando prohibimos la actualización o eliminación de un dato referenciado.

Sin las opciones DELETE CASCADE o ON DELETE SET NULL, las filas en la tabla padre no pueden ser eliminadas si estas son referenciadas en la tabla hija.

Constraint CHECK

El *constraint CHECK* define una condición que para cada fila debe satisfacerse. La condición puede utilizarse de manera similar a las condiciones de una consulta, con las siguientes excepciones:

- Las referencias a columnas CURRVAL, NEXTVAL, LEVEL y ROWNUM
- Llamadas a funciones SYSDATE, UID, USER y USERENV
- Consultas que hagan referencia a valores en otras filas de la misma tabla

Una columna puede tener múltiples *constraints CHECK* las cuales pueden reverenciarse en la definición de la columna. No se tienen limites en el número de *constraints CHECK* cuando se define una columna.

```
CREATE TABLE employees
(
...
salary NUMBER(8,2) CONSTRAINT emp_salary_min
CHECK (salary > 0),
...
)
```


Añadiendo un constraint

```
ALTER TABLE table
ADD [CONSTRAINT constraint] type (column);
```

Se pueden añadir *constraints* a tablas existentes con el uso de la sentencia ALTER TABLE y la cláusula ADD.

Sintaxis:

<i>table</i>	es el nombre de la tabla
<i>constraint</i>	es el nombre del <i>constraint</i>
<i>type</i>	es el tipo de <i>constraint</i>
<i>column</i>	es el nombre de la columna afectada por el <i>constraint</i>

En la sintaxis el nombre del *constraint* es opcional, aunque es recomendado. Si no se le especifica un nombre al *constraint*, el sistema genera uno.

Normas a seguir

- Se puede añadir, eliminar, habilitar o deshabilitar un *constraint*, pero no se puede modificar su estructura.
- Se puede añadir un *constraint NOT NULL* a una columna existente con el uso de la cláusula MODIFY de la sentencia ALTER TABLE

Nota: se puede definir una columna NOT NULL solamente si la tabla se encuentra vacía o si la columna tiene un valor para cada fila.

```
ALTER TABLE      employees
ADD CONSTRAINT    emp_manager_fk
FOREIGN KEY(manager_id)
REFERENCES employees(employee_id);
Table altered.
```

En el ejemplo anterior se crea un *constraint FOREIGN KEY* en la tabla EMPLOYEES. El *constraint* asegura que un jefe exista como un empleado válido en la tabla EMPLOYEES.

Eliminando un constraint

Para eliminar un *constraint*, se puede identificar el nombre del *constraint* desde las vistas del diccionario de datos USER_CONSTRAINTS y USER_CONS_COLUMNS. Entonces se usa la sentencia ALTER TABLE con la cláusula DROP. La opción CASCADE de la cláusula DROP ocasiona que todos los *constraints* dependientes sean borrados.

```
ALTER TABLE      employees
DROP CONSTRAINT    emp_manager_fk;
Table altered.
```

```
ALTER TABLE departments
DROP PRIMARY KEY CASCADE;
Table altered.
```

Sintaxis

```
ALTER TABLE table
DROP PRIMARY KEY | UNIQUE (column) |
CONSTRAINT constraint [CASCADE];
```

Donde:

<i>table</i>	es el nombre de la tabla
<i>column</i>	es el nombre de la columna afectada por el <i>constraint</i>
<i>constraint</i>	es el nombre del <i>constraint</i>

Cuando se elimina un *constraint*, deja de estar disponible en el diccionario de datos.

Deshabilitando un constraint

```
ALTER TABLE employees
DISABLE CONSTRAINT emp_emp_id_pk CASCADE;
Table altered.
```

Se puede deshabilitar un *constraint* sin eliminarlo o recrearlo con el uso de la sentencia ALTER TABLE y la cláusula DISABLE.

Sintaxis

```
ALTER TABLE table
DISABLE CONSTRAINT constraint [CASCADE];
```

Donde:

<i>table</i>	es el nombre de la tabla
<i>constraint</i>	es el nombre del <i>constraint</i>

Normas a seguir

- Se puede usar la cláusula DISABLE en las sentencias CREATE TABLE y ALTER TABLE
- La cláusula CASCADE deshabilita las dependencias de los *constraints* de integridad
- Deshabilitar un *constraint* único o *llave primaria*, elimina el índice único

Habilitando un constraint

```
ALTER TABLE      employees
ENABLE CONSTRAINT  emp_emp_id_pk;
Table altered.
```

Se puede habilitar un *constraint* sin eliminarlo o recrearlo utilizando la sentencia ALTER TABLE con la cláusula ENABLE.

Sintaxis

```
ALTER   TABLE      table
ENABLE  CONSTRAINT  constraint;
```

Donde:

table es el nombre de la tabla
constraint es el nombre del *constraint*

Normas a seguir

- Si se habilita un *constraint*, este aplica a todos los datos de la tabla. Todos los datos de la tabla deben satisfacer el *constraint*.
- Si se habilita un *constraint PRIMARY KEY* o *UNIQUE key*, se crea un índice *único* o de *llave primaria* automáticamente
- Se puede usar la cláusula ENABLE en las sentencias CREATE TABLE y ALTER TABLE
- Habilitar un *constraint de llave primaria* que ha sido deshabilitado con la opción CASCADE no habilita ninguna *llave foránea* dependiente de la *llave primaria*.

Constraints en cascada

La siguiente sentencia ilustra el uso de la cláusula CASCADE CONSTRAINTS. Considerando que la tabla TEST1 es creada como sigue:

```
CREATE TABLE test1 (
  pk NUMBER PRIMARY KEY,
  fk NUMBER,
  col1 NUMBER,
  col2 NUMBER,
  CONSTRAINT fk_constraint FOREIGN KEY (fk) REFERENCES test1,
  CONSTRAINT ck1 CHECK (pk > 0 and col1 > 0),
  CONSTRAINT ck2 CHECK (col2 > 0));
```

El siguiente error es obtenido

```
ALTER TABLE test1 DROP (pk); -- pk is a parent key
ALTER TABLE test1 DROP (col1); -- col1 is referenced by multicolumn constraint ck1
```

Cascading Constraints

Example:

```
ALTER TABLE test1  
DROP (pk) CASCADE CONSTRAINTS;  
Table altered.
```

```
ALTER TABLE test1  
DROP (pk, fk, col1) CASCADE CONSTRAINTS;  
Table altered.
```

ORACLE

Consideremos la siguiente sentencia que elimina la columna PK, el *constraint de llave primaria*, el *constraint foráneo fk_constraint* y el *constraint check CK1*:

```
ALTER TABLE test1 DROP (pk) CASCADE CONSTRAINTS;
```

Si todas las columnas referenciadas por los *constraints* definidos en las columnas eliminadas son también borradas, entonces la cláusula CASCADE CONSTRAINTS no es requerida. Por ejemplo, considerando que no existen otros *constraints* referenciados de otras tablas a la columna PK, es correcta la siguiente sentencia sin la cláusula CASCADE CONSTRAINT:

```
ALTER TABLE test1 DROP (pk, fk, col1);
```

Viewing Constraints

Query the `USER_CONSTRAINTS` table to view all constraint definitions and names.

```
SELECT  constraint_name, constraint_type,
        search_condition
FROM    user_constraints
WHERE   table_name = 'EMPLOYEES';
```

CONSTRAINT_NAME	C	SEARCH_CONDITION
EMP_LAST_NAME_NN	C	"LAST_NAME" IS NOT NULL
EMP_EMAIL_NN	C	"EMAIL" IS NOT NULL
EMP_HIRE_DATE_NN	C	"HIRE_DATE" IS NOT NULL
EMP_JOB_NN	C	"JOB_ID" IS NOT NULL
EMP_SALARY_MIN	C	salary > 0
EMP_EMAIL_UK	U	
...		

ORACLE

Viendo constraints

Después de crear una tabla, puedes confirmar su existencia con el uso del comando `DESCRIBE`. El único *constraint* que puedes verificar es el *constraint NOT NULL*. Para ver todos los *constraints* de tu tabla, consulta la tabla `USER_CONSTRAINTS`.

En el ejemplo anterior se muestran los *constraints* de la tabla `EMPLOYEES`.

Nota: Los *constraints* que no son nombrados por el propietario de la tabla reciben un nombre asignado por el servidor de Oracle. En el tipo de *constraint*, C indica *CHECK*, P *PRIMARY KEY*, R integridad referencial y U *llave única*. Note que el *constraint NOT NULL* es realmente un *constraint CHECK*.

Viewing the Columns Associated with Constraints

View the columns associated with the constraint names in the USER_CONS_COLUMNS view.

```
SELECT constraint_name, column_name
FROM   user_cons_columns
WHERE  table_name = 'EMPLOYEES';
```

CONSTRAINT_NAME	COLUMN_NAME
EMP_DEPT_FK	DEPARTMENT_ID
EMP_EMAIL_NN	EMAIL
EMP_EMAIL_UK	EMAIL
EMP_EMP_ID_FK	EMPLOYEE_ID
EMP_HIRE_DATE_NN	HIRE_DATE
EMP_JOB_FK	JOB_ID
EMP_JOB_NN	JOB_ID
...	

ORACLE

Se puede ver el nombre de las columnas involucradas en un *constraint* consultando la vista del diccionario de datos USER_CONS_COLUMNS. Esta vista es especialmente útil para los *constraints* que utilizan nombres asignados por el sistema.

Resumen

En este capítulo se ha revisado como el servidor de Oracle usa *constraints* para prevenir el registro de datos inválidos a las tablas. Así como su implementación en sentencias DDL.

Los siguientes tipos de *constraints* son válidos:

- NOT NULL
- UNIQUE
- PRIMARY KEY
- FOREIGN KEY
- CHECK

Se puede consultar la tabla USER_CONSTRAINTS para ver la definición de todos los *constraints* y sus nombres.